# Lire un message d'erreur

## Où regarder, ce que dit chaque partie, quoi faire

Kevin Bardoux

## Table des matières

La méthode, pour tous les outils ..... 2
Node.js (semaine 2) ..... 2
Une variable qui n'existe pas : `ReferenceError` ..... 2
Une opération impossible : `TypeError` ..... 3
Une faute de syntaxe : `SyntaxError` ..... 3
Un fichier introuvable : `Cannot find module` ..... 3
Un test rouge : `node --test` ..... 4
Tout le fichier de tests échoue : `tests 1`, `fail 1` ..... 4
Le test ne trouve pas ma fonction : `is not a function`, `undefined` ..... 5
Le vocabulaire des messages de test (semaine 5) ..... 6
ESLint (semaine 2) ..... 9
ESLint ne souligne rien dans VS Code ..... 9
npm parle de vulnérabilités ..... 10
Git (semaine 4) ..... 11
Pas dans un dépôt ..... 11
`git log` ne montre rien ..... 11
Un conflit de fusion ..... 11
Un envoi refusé ..... 12
Un avertissement de fin de ligne (Windows) ..... 12
Sources ..... 13

Un message d'erreur n'est pas un reproche : c'est une indication précise, qui dit **quoi**, **où** et souvent **pourquoi**. Cette fiche montre

de vrais messages, outil par outil. Elle grandit avec la formation : une section y entre avec la semaine de son outil.

## La méthode, pour tous les outils

1. **Lisez le type** : le premier mot important (`TypeError`, `error`, `fatal`, `CONFLICT`…).
2. **Lisez le message** : la phrase qui suit, mot par mot. Cherchez les mots inconnus.
3. **Trouvez l'endroit** : le fichier, la ligne, parfois la colonne.
4. **Corrigez une chose à la fois**, puis relancez.

C'est la démarche de débogage de la semaine 3 : lire, isoler, tester, corriger. Les chemins des fichiers affichés ici viennent d'un ordinateur sous Linux ; sous Windows, ils commencent par `C:/Users/…`. Certains outils affichent leurs messages en français : le sens et la place de chaque partie restent les mêmes.

## Node.js (semaine 2)

### Une variable qui n'existe pas : `ReferenceError`

```
file:///…/dwwm/semaine-02/ref.js:1
console.log(prix);
            ^

ReferenceError: prix is not defined
    at file:///…/dwwm/semaine-02/ref.js:1:13
```

- **Où** : la première ligne donne le fichier et la ligne (`ref.js:1`) ; le `^` montre l'endroit exact.
- **Type** : `ReferenceError`, une variable utilisée alors qu'elle n'existe pas (MDN Web Docs, s. d.-a).
- **Message** : `prix is not defined`, « `prix` n'est pas défini ».
- **Quoi faire** : vérifiez l'orthographe du nom, et que la variable est déclarée (`const`, `let`) **avant** cette ligne, dans le même bloc.

### Une opération impossible : `TypeError`

```
file:///…/dwwm/semaine-02/type.js:2
console.log(word.length);
                 ^

TypeError: Cannot read properties of undefined (reading
'length')
    at file:///…/dwwm/semaine-02/type.js:2:18
```

- **Type** : `TypeError`, une opération impossible sur une valeur du mauvais type (MDN Web Docs, s. d.-b).
- **Message** : « impossible de lire les propriétés de `undefined` (en lisant `length`) » : `word` vaut `undefined`.
- **Quoi faire** : cherchez pourquoi la valeur vaut `undefined` (un argument oublié, une variable jamais remplie), puis traitez ce cas d'abord (style défensif).

### Une faute de syntaxe : `SyntaxError`

```
file:///…/dwwm/semaine-02/syn.js:1
console.log('bonjour';
            ^^^^^^^^^

SyntaxError: missing ) after argument list
```

- **Type** : `SyntaxError`, le code ne respecte pas la grammaire de JavaScript (MDN contributors, s. d.) : le programme ne démarre même pas.
- **Message** : « il manque `)` après la liste des arguments ».
- **Quoi faire** : regardez la ligne indiquée **et** la précédente ; comptez les parenthèses, les accolades et les guillemets ouverts et fermés.

### Un fichier introuvable : `Cannot find module`

```
Error: Cannot find module '/…/dwwm/semaine-02/
absent.js'
```

- **Message** : Node ne trouve pas le fichier demandé (OpenJS Foundation, s. d.-a).
- **Quoi faire** : tapez `pwd`, puis `ls` : êtes-vous dans le bon dossier ? Le nom est-il exact, extension comprise (pas `absent.js.txt`) ?

Sous chaque erreur, les lignes qui commencent par `at` forment la **pile d'appels** : la première ligne qui mentionne **votre** fichier est la plus utile ; les lignes `node:internal/…` concernent Node lui-même.

### Un test rouge : `node --test`

```
✖ affiche Hello, world!
  AssertionError [ERR_ASSERTION]: Expected values to be
strictly equal:
  + actual - expected

  + 'À faire'
  - 'Hello, world!'
```

- **Où** : le nom du test qui échoue, après ✖.
- **`+ actual`** : ce que votre programme affiche ou renvoie ; **`- expected`** : ce qu'il devrait afficher (OpenJS Foundation, s. d.-c).
- **Quoi faire** : comparez les deux lignes caractère par caractère (espaces, majuscules, ponctuation), corrigez, relancez.

Les mots anglais de ces messages sont traduits dans le vocabulaire des messages de test, plus bas.

### Tout le fichier de tests échoue : `tests 1`, `fail 1`

```
file:///…/aire.js:2
  let width = 3;
      ^

SyntaxError: Identifier 'width' has already been
declared
    at …
✖ aire.test.js
ℹ tests 1
```

```
ℹ pass 0
ℹ fail 1
```

- **Où** : après ✖, le nom du **fichier** de tests, pas celui d'un test ; et `tests 1` alors que ce fichier en contient plusieurs.
- **Message** : votre fichier ne s'est pas chargé, aucun test n'a pu tourner (OpenJS Foundation, s. d.-c). La cause est écrite au-dessus : une erreur de syntaxe (ici, un paramètre déclaré une deuxième fois), ou une ligne placée hors d'une fonction qui plante au chargement.
- **Quoi faire** : lancez `node votre-fichier.js` pour voir l'erreur seule ; lisez la ligne indiquée et le `^` ; essayez vos fonctions dans un fichier `essai.js` séparé, jamais dans le fichier testé.

### Le test ne trouve pas ma fonction : `is not a function`, `undefined`

```
✖ withTax ajoute 20 %
  TypeError: withTax is not a function
```

```
✖ TAX_RATE est exportée et vaut 0.2
  AssertionError [ERR_ASSERTION]: Expected values to be
strictly equal:

  undefined !== 0.2
```

```
file:///…/essai.js:1
import { withTax } from './l7-exporter.js';
         ^^^^^^^
SyntaxError: The requested module './l7-exporter.js'
does not provide an export named 'withTax'
```

- **Où** : tous les tests de la même fonction échouent, avec le même message ; le nom cité (`withTax`, `TAX_RATE`) est celui de votre fonction ou de votre constante.
- **Message** : le fichier de tests importe votre fichier, mais n'y trouve pas ce nom : il n'est pas **exporté**, ou il est écrit

autrement. Un nom introuvable vaut `undefined` : appeler `undefined` donne `is not a function` ; comparer `undefined` à la valeur attendue donne `undefined !== 0.2`. Avec un `import { … } from …` écrit en haut d'un fichier, comme dans votre `essai.js`, Node refuse même de démarrer : `does not provide an export named 'withTax'` (« ne fournit pas d'export nommé ») (OpenJS Foundation, s. d.-b).

- **Quoi faire** : dans votre fichier, écrivez `export` devant la déclaration, puis relancez les tests :

```
export const TAX_RATE = 0.2;

export function withTax(prices) {
  return prices.map((price) => price * (1 +
TAX_RATE));
}
```

Vérifiez aussi que le nom est écrit exactement comme dans le test, majuscules comprises. Indice dans VS Code : ESLint souligne une fonction ou une constante jamais utilisée ni exportée (`'withTax' is defined but never used`, règle `no-unused-vars`).

## Le vocabulaire des messages de test (semaine 5)

Les messages des tests sont en anglais. Ces quelques mots suffisent pour les lire ; ils reviennent avec Vitest, en semaine 10.

| Mot | Traduction | Où vous le voyez |
|---|---|---|
| *test*, *pass*, *fail* | test, réussi, échoué | `ℹ tests 4`, `ℹ pass 3`, `ℹ fail 1` : le bilan, en bas |
| *assertion* | affirmation vérifiée par le test | `AssertionError` : une affirmation du test est fausse |

| Mot | Traduction | Où vous le voyez |
|---|---|---|
| *actual* | obtenu : ce que votre code a renvoyé | `+ actual`, et les lignes qui commencent par `+` |
| *expected* | attendu : ce que le test demandait | `- expected`, et les lignes qui commencent par `-` |
| *strictly equal* | strictement égal (`===`) | comparaison de deux valeurs simples (nombre, texte) |
| *deep-equal* | égal en profondeur : même contenu | comparaison de deux tableaux ou objets, élément par élément |
| *falsy value* | valeur fausse (`false`, `0`, `''`, `null`, `undefined`) | le test attendait une valeur vraie |
| *did not match* | ne correspond pas | le texte ne contient pas le motif attendu |
| *is missing (from)* | est absent (de) | `--couleur-texte is missing from :root` : la déclaration manque dans `:root` |
| *no*, *not*, *without* | aucun, n'est pas, sans | `no min-width media query`, `margin-left is not auto`, `image without alt` |
| *fewer than* | moins de | `fewer than three --espace-… variables` |
| *outside the scale* | hors de l'échelle | la valeur n'est pas une variable de |

| Mot | Traduction | Où vous le voyez |
| --- | --- | --- |
| | | l'échelle (`--espace-…`, `--texte-…`) |
| *hard-coded* | écrit en dur, à la main | une couleur écrite directement, au lieu d'une `var(--couleur-…)` |
| *exactly one*, *must* | un seul, exactement ; doit | `exactly one element must have the accent background` |
| *found*, *unitless*, *running text* | trouvé, sans unité, texte courant | `no spacing found`, `line-height (unitless)`, `text-align: center on running text` |

Trois messages réels de `node --test` (Node 24, chemins raccourcis), avec leur lecture :

```
AssertionError [ERR_ASSERTION]: Expected values to be
strictly equal:

15 !== 16
```

- **Lecture** : votre code a renvoyé `15` (*actual*), le test attendait `16` (*expected*) ; `!==` : « n'est pas égal à ».

```
AssertionError [ERR_ASSERTION]: Expected values to be
strictly deep-equal:
+ actual - expected

  [
    2,
    4,
-   6
  ]
```

- **Lecture** : les deux tableaux diffèrent ; la ligne `- 6` est attendue mais absente de votre résultat : il manque le `6`.

```
AssertionError [ERR_ASSERTION]: The expression
evaluated to a falsy value:

  assert.ok(false)
```

- **Lecture** : le test vérifiait qu'une valeur est vraie (par exemple `Number.isNaN(…)`) ; elle est fausse.

## ESLint (semaine 2)

```
 8:1   error  Unexpected var, use let or const instead
no-var
12:15  error  Expected '===' and instead saw '=='
eqeqeq
17:13  error  'mesage' is not defined
no-undef

✖ 5 problems (5 errors, 0 warnings)
```

- **Où** : `ligne:colonne` en début de ligne (`12:15` : ligne 12, colonne 15).
- **Message** : ce qui ne va pas, en anglais.
- **Règle** : le dernier mot (`eqeqeq`) ; il mène à sa page de documentation (ESLint, s. d.).
- **Quoi faire** : corrigez une alerte à la fois, en partant du haut, puis relancez `npx eslint .`.

### ESLint ne souligne rien dans VS Code

Pas de message ici : c'est l'absence de vague rouge qui alerte, alors que le fichier contient une erreur (par exemple `var x = 1;`). Vérifiez dans l'ordre :

1. **`npm install` a-t-il été lancé dans ce dossier ?** Un dossier `node_modules` doit être à côté de `package.json`. Sans lui, `npm run lint` dans le terminal se termine par une ligne qui dit que la commande `eslint` est introuvable (son libellé dépend du

système ; sous Linux : `sh: 1: eslint: not found`). Lancez `npm install` dans le dossier de la semaine.

2. **Le bon dossier est-il ouvert ?** VS Code doit avoir ouvert le dossier de la semaine (celui qui contient `eslint.config.js`), pas `dwwm` tout entier ni un fichier seul : *Fichier*, *Ouvrir le dossier…*.
3. **L'extension ESLint est-elle installée et activée ?** Vue Extensions (`Ctrl+Shift+X` (windows), `Cmd+Shift+X` (mac)) : cherchez ESLint (éditeur Microsoft) ; le bouton doit proposer *Désactiver*, pas *Installer* ni *Activer*.
4. **Toujours rien ?** Tapez `npx eslint nom-du-fichier.js` dans le terminal. S'il affiche les erreurs, ESLint marche et c'est l'éditeur qui ne l'a pas encore chargé : fermez puis rouvrez VS Code. S'il n'affiche rien non plus, appelez le formateur.

## npm parle de vulnérabilités

```
5 moderate severity vulnerabilities

To address all issues (including breaking changes),
run:
  npm audit fix --force
```

- **Ce que ça veut dire** : à la fin de `npm install`, npm compare les paquets installés à une liste de failles connues (npm Docs, s. d.). Ici, une faille a été publiée dans un paquet dont ESLint a besoin. Elle concerne l'outil installé sur votre poste, pas votre code, et `npm install` a bien fonctionné.
- **Ce qu'il ne faut pas faire** : lancer `npm audit fix --force` sans comprendre. *including breaking changes* : « y compris des changements qui cassent ». Pour faire disparaître le message, npm peut installer une tout autre version d'un paquet : en octobre 2026, il proposait ESLint 4.0.0, une version de 2017, et VS Code ne soulignait plus vos erreurs.

- **Quoi faire** : rien pendant un TP ; signalez-le au formateur, qui vérifie la faille et met à jour le kit si besoin. `npm audit` affiche le détail : le paquet en cause et la version corrigée.
- **Pour le savoir** : sur un vrai projet, on lit chaque faille avant de corriger (semaine 2 : les risques des paquets). `npm audit fix`, sans `--force`, ne fait que des mises à jour compatibles.

# Git (semaine 4)

## Pas dans un dépôt

```
fatal: not a git repository (or any of the parent
directories): .git
```

- **Message** : le dossier courant n'est pas un dépôt Git, ni aucun dossier parent.
- **Quoi faire** : tapez `pwd` ; placez-vous dans le dossier du projet (`cd …`), ou créez le dépôt avec `git init` s'il n'existe pas encore.

## `git log` ne montre rien

```
fatal: votre branche actuelle 'main' ne contient encore
aucun commit
```

En anglais : `fatal: your current branch 'main' does not have any commits yet`. Avec `--graph --all`, `git log` n'affiche rien du tout, sans message.

- **Message** : le dossier est bien un dépôt Git, mais aucun commit n'y est encore enregistré.
- **Quoi faire** : tapez `pwd` : êtes-vous dans le bon dossier (par exemple `~/dwwm/git-journal`) ? Un `git init` lancé dans un autre dossier crée un second dépôt, vide. Dans le bon dossier, tapez `git status` : des fichiers en rouge ou en vert attendent `git add`, puis `git commit -m "…"`. Si le message est `not a git repository`, voir « Pas dans un dépôt ».

## Un conflit de fusion

```
Auto-merging journal.md
CONFLICT (content): Merge conflict in journal.md
```

```
Automatic merge failed; fix conflicts and then commit
the result.
```

Le même message, quand Git s'affiche en français :

```
Fusion automatique de journal.md
CONFLIT (contenu) : Conflit de fusion dans journal.md
La fusion automatique a échoué ; réglez les conflits et
validez le résultat.
```

- **Où** : le fichier en conflit (`journal.md`).
- **Message** : les deux branches ont modifié les mêmes lignes ; Git ne choisit pas.
- **Quoi faire** : ouvrez le fichier, gardez le bon texte entre les marqueurs `<<<<<<<`, `=======` et `>>>>>>>`, effacez les marqueurs, puis `git add` et `git commit` (GitHub, s. d.-b).

## Un envoi refusé

```
 ! [rejected]        HEAD -> main (fetch first)
error: failed to push some refs to '…'
hint: Updates were rejected because the remote contains
work that you do
hint: not have locally.
```

- **Message** : le dépôt distant contient des commits que vous n'avez pas.
- **Quoi faire** : récupérez-les d'abord avec `git pull`, réglez un éventuel conflit, puis relancez `git push` (Scott Chacon et Ben Straub, s. d.). Jamais de `git push --force` sans l'accord du formateur.

## Un avertissement de fin de ligne (Windows)

```
warning: in the working copy of 'notes.md', LF will be
replaced by CRLF the next time Git touches it
```

- **Où** : après `git add`, sous Windows, pour un fichier créé dans Git Bash (par exemple avec `echo`).

- **Message** : ce n'est pas une erreur. Windows termine les lignes autrement que macOS et Linux (CRLF au lieu de LF) ; Git convertit tout seul (GitHub, s. d.-a). Le fichier est bien ajouté.
- **Quoi faire** : rien ; continuez avec `git commit`. Cet avertissement reste en anglais, même quand Git s'affiche en français.

## Sources


ESLint. s. d. « Getting Started with ESLint ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/use/getting-started.

GitHub. s. d.-a. « Configuring Git to handle line endings ». Consulté le 17 octobre 2026. https://docs.github.com/en/get-started/git-basics/configuring-git-to-handle-line-endings.

GitHub. s. d.-b. « Resolving a merge conflict using the command line ». Consulté le 26 septembre 2026. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line.

MDN contributors. s. d. « SyntaxError — JavaScript | MDN ». Consulté le 3 octobre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/SyntaxError.

MDN Web Docs. s. d.-a. « ReferenceError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/ReferenceError.

MDN Web Docs. s. d.-b. « TypeError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/TypeError.

npm Docs. s. d. « npm-audit ». Consulté le 2 octobre 2026. https://docs.npmjs.com/cli/v11/commands/npm-audit.

OpenJS Foundation. s. d.-a. « Errors — Node.js Documentation ». Consulté le 3 octobre 2026. https://nodejs.org/api/errors.html.

OpenJS Foundation. s. d.-b. « Modules: ECMAScript modules — Node.js Documentation ». Consulté le 30 septembre 2026. https://nodejs.org/api/esm.html.

OpenJS Foundation. s. d.-c. « Test runner | Node.js Documentation ». Consulté le 26 septembre 2026. https://nodejs.org/api/test.html.

Scott Chacon et Ben Straub. s. d. « Pro Git : Travailler avec des dépôts distants ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-bases-de-Git-Travailler-avec-des-d%C3%A9p%C3%B4ts-distants.